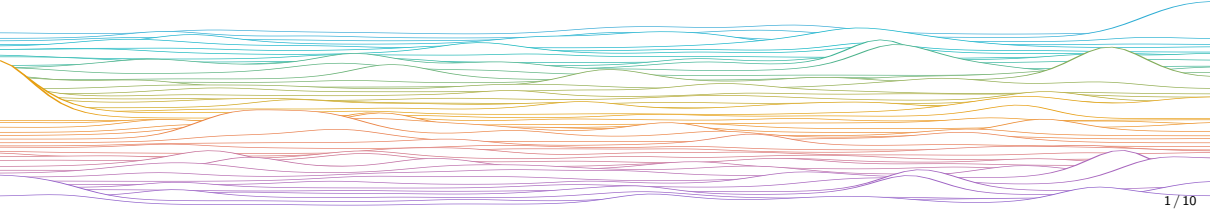
Gradient descent

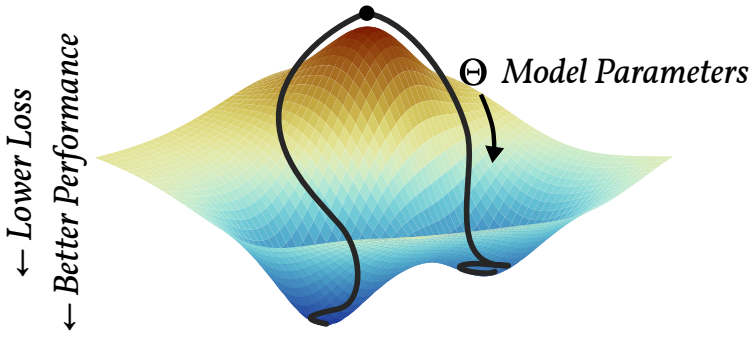
Computational Neuroscience
University of Bristol

M Rule

Learning outcomes:

- Review terminology and mathematical notation for gradient descent optimization
- Be prepared to interpret certain biological learning rules as gradient-descent optimization




← Lower Loss
← Better Performance
Θ Model Parameters

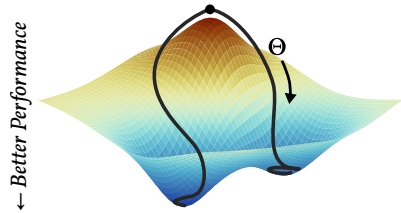
Gradient descent

For training pair $(\boldsymbol{x}, \boldsymbol{y})$ define

$\hat{\boldsymbol{y}} = f(\boldsymbol{x}, \Theta)$ **Prediction**

$\Theta = \{\theta_1, \theta_2, \dots\}$ **Parameters**

$\mathcal{L}(\hat{\boldsymbol{y}}, \boldsymbol{y})$ **Loss** (make small)



Adjust Θ to **descend loss gradient:**

$$\Theta_{t+1} \leftarrow \Theta_t - \eta \cdot \nabla_\Theta \mathcal{L}[f(\boldsymbol{x}, \Theta), \boldsymbol{y}],$$

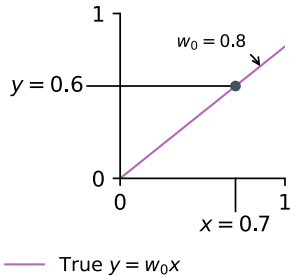
where $\eta > 0$ is the learning rate.

"empirical risk minimization"

- Minimize average loss $\langle \mathcal{L}[f(\boldsymbol{x}_i, \Theta), \boldsymbol{y}_i] \rangle_{i \in \text{training data}}$ over training examples

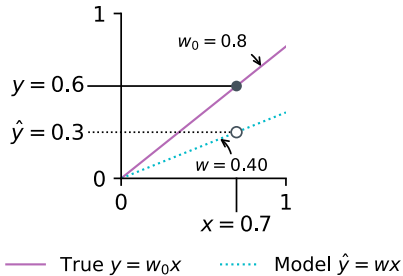
loss/cost/objective/penalty/(negative)(log)likelihood, etc.

Given sample (x, y) from a linear process $y = w_0 x$ with unknown w_0,



Given sample (x, y) from a linear process $y = w_0 x$ with unknown w_0,

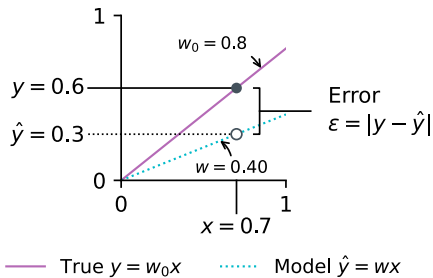
learn $f : x \mapsto y$ $\hat{y}(x, w) = wx$ **prediction**



Given sample (x, y) from a linear process $y = w_0 x$ with unknown w_0,

learn $f : x \mapsto y$ $\quad \hat{y}(x, w) = wx$ **prediction**

Pick loss function $\quad \mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$ **loss**



Given sample (x, y) from a linear process $y = w_0 x$ with unknown w_0,

learn $f : x \mapsto y$

$\hat{y}(x, w) = wx$ — **prediction**

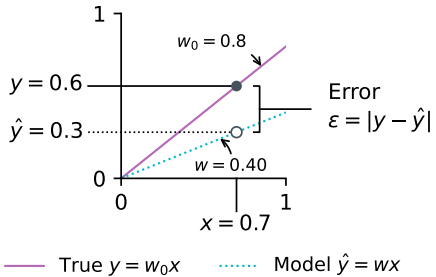
Pick loss function

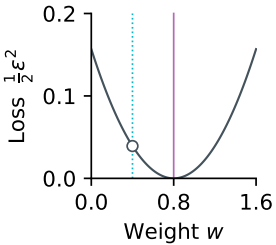
$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$ — **loss**

Adjust w to decrease $\mathcal{L}$

(η: learning rate)

$w_{t+1} \leftarrow w_t - \eta \cdot \frac{\mathrm{d}\,\mathcal{L}}{\mathrm{d}\,w}\Big|_{w_t}$ — **gradient descent**





Given sample (x, y) from a linear process $y = w_0 x$ with unknown w_0,

learn $f : x \mapsto y$

$\hat{y}(x, w) = wx$ — **prediction**

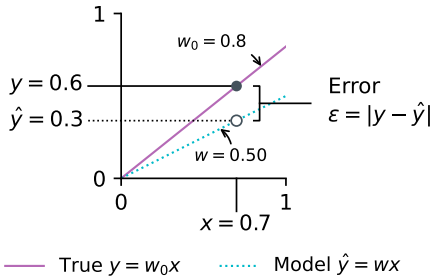
Pick loss function

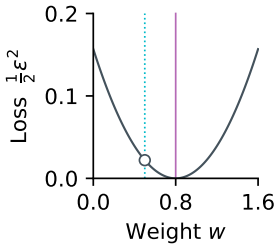
$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$ — **loss**

Adjust w to decrease $\mathcal{L}$

(η: learning rate)

$w_{t+1} \leftarrow w_t - \eta \cdot \frac{\mathrm{d}\,\mathcal{L}}{\mathrm{d}\,w}\Big|_{w_t}$ — **gradient descent**





Given sample (x, y) from a linear process $y = w_0 x$ with unknown w_0,

learn $f : x \mapsto y$

$\hat{y}(x, w) = wx$ — **prediction**

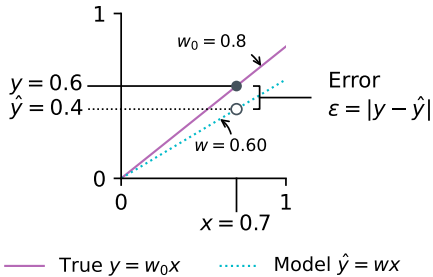
Pick loss function

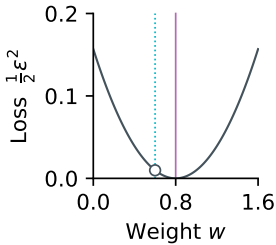
$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$ — **loss**

Adjust w to decrease $\mathcal{L}$

(η: learning rate)

$w_{t+1} \leftarrow w_t - \eta \cdot \frac{\mathrm{d}\,\mathcal{L}}{\mathrm{d}\,w}\Big|_{w_t}$ — **gradient descent**





Given sample (x, y) from a linear process $y = w_0 x$ with unknown w_0,

learn $f : x \mapsto y$

$\hat{y}(x, w) = wx$ — **prediction**

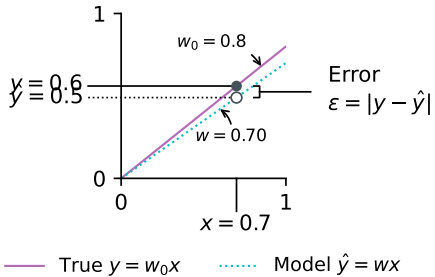
Pick loss function

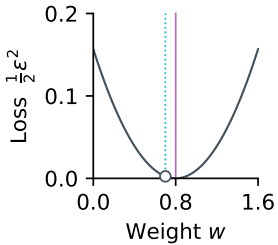
$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$ — **loss**

Adjust w to decrease $\mathcal{L}$

(η: learning rate)

$w_{t+1} \leftarrow w_t - \eta \cdot \frac{\mathrm{d}\,\mathcal{L}}{\mathrm{d}\,w}\Big|_{w_t}$ — **gradient descent**





Given sample (x, y) from a linear process $y = w_0 x$ with unknown w_0,

learn $f : x \mapsto y$

Pick loss function
Adjust w to decrease $\mathcal{L}$
(η: learning rate)

$$\hat{y}(x, w) = wx$$

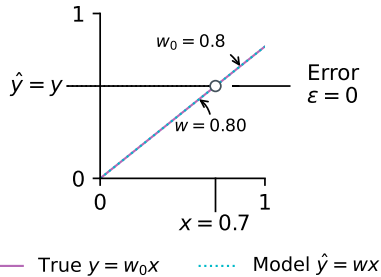
$$\mathcal{L}(\hat{y}, y) = \tfrac{1}{2}(y - \hat{y})^2$$

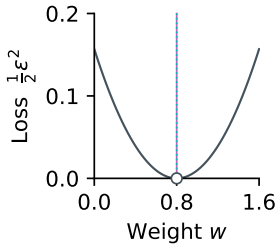
$$w_{t+1} \leftarrow w_t - \eta \cdot \frac{\mathrm{d}\,\mathcal{L}}{\mathrm{d}\,w}\Big|_{w_t}$$

prediction

loss

gradient descent





Given sample (x, y) from a linear process $y = w_0 x$ with unknown w_0,

learn $f : x \mapsto y$

$\hat{y}(x, w) = wx$ — **prediction**

Pick loss function

$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(y - \hat{y})^2$ — **loss**

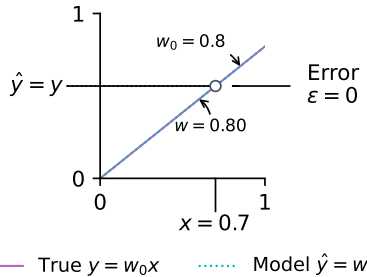
Adjust w to decrease $\mathcal{L}$

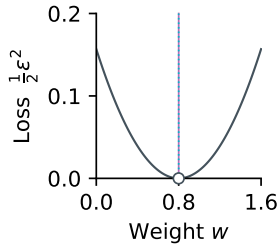
(η: learning rate)

$w_{t+1} \leftarrow w_t - \eta \cdot \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}w}\Big|_{w_t}$ — **gradient descent**

Chain rule

$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}w} = \frac{\mathrm{d}}{\mathrm{d}w}[\mathcal{L} \circ \hat{y}] = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\hat{y}} \frac{\mathrm{d}\hat{y}}{\mathrm{d}w} = -(y - \hat{y})x$ → ***backpropagation***





work through a simple example by hand

Select flavours of gradient descent

Say you have

- A dataset of input–output training sample pairs $\{\boldsymbol{x}, y^*\}_i$.
- A per-sample loss $\mathcal{L}(y_i^*, \boldsymbol{x}_i; \mathbf{w})$

Online gradient descent

- Update $\Delta\mathbf{w} = -\eta \cdot \partial_{\mathbf{w}} \mathcal{L}(y_i^*, \boldsymbol{x}_i)$ for each pair as they arrive

Batch gradient descent

- Average over multiple samples (in example below, all data), before updating
- Update $\Delta\mathbf{w} = -\eta \cdot \langle \partial_{\mathbf{w}} \mathcal{L}(y_i^*, \boldsymbol{x}_i) \rangle_i$

Gradient flow

- Take the $\eta \to 0$ limit to view learning as an ODE
- E.g. (batch) gradient flow $\dot{\mathbf{w}} = -\langle \partial_{\mathbf{w}} \mathcal{L}(y_i^*, \boldsymbol{x}_i) \rangle_i$

There may be an exam problem using gradient flow to analyze the stability of a learning rule

Gradient flow

- Take the $\eta \to 0$ limit to view learning as an ODE
- E.g. (batch) gradient flow $\dot{\mathbf{w}} = -\langle \partial_{\mathbf{w}} \mathcal{L}(y_i^*, \boldsymbol{x}_i) \rangle_i$

E.g. from these slides

$$\dot{w} = \langle (y - \hat{y})x \rangle = \langle (y - wx)x \rangle = \langle yx \rangle - w\langle x^2 \rangle$$

Where does it stand still (fixed point)? Set $\dot{w} = 0$

$$\langle yx \rangle = w\langle x^2 \rangle \quad \Rightarrow \quad w = \langle yx \rangle \langle x^2 \rangle^{-1}$$

(closed form least-squares solution for linear regression through the origin)

We'll work through a version with a more biological learning rule later

end

page left blank for notes

page left blank for notes